Lab 7: SDN Application Development - Shortest Path Routing, Firewall, and VM Migration in a Clos Data Center

Course: **FSCT 8540 – Network Security**
Instructor: Dr. Maryam R. Aliabadi
Lab Duration: 5 hours

---

1. Introduction

In this assignment you will acquire a practical view of the concepts of **network virtualization** and **cloud management** in the context of data center environments. In particular, you will write **SDN applications** on top of an **OpenFlow controller (POX)** to implement routing, enforce firewall policies and provide support for **Virtual Machine migration** over a data center-like topology. This controller will essentially act as a network controller for a small cloud facility.

In general, network virtualization permits systems to share physical network resources so as to form diverse **virtual networks (slices)** on top of the infrastructure. Each of these slices has, for example, its dedicated bandwidth allocation, broadcast domain, etc.. This concept simplifies network management and increases the resource utilization of the physical infrastructure, allowing a cloud provider to amortize the costs of the underlying network. Mechanisms related to network virtualization are commonly used in data centers to orchestrate their multiple applications servers—tenants. SDN in general and OpenFlow in particular can be used to implement such mechanisms, as you have learned in the lecture.

---

2. Emulate a Data Center and Manage it via a Cloud Network Controller

In this exercise, your first task is to create a multi-rooted **tree-like (Clos) topology** in **Mininet** to emulate a data center (see Figures 1 and 2). Your second task is to implement specific SDN applications on top of the network controller in order to **orchestrate multiple network tenants** within a data center environment, in the context of network virtualization and management.

In Figure 1, we see a sample data center network design with a number of **Virtual Machines (VMs)** being hosted by distinct VM end-host servers. For simplicity, we assume that each

server hosts exactly one VM instance. In Figure 1, each set of VMs with the same color belongs to separate data center tenants ; tenants may represent different application environments (different colors respectively) that the VMs host. For example, one tenant may be an enterprise client running its big data analytics, while another tenant may be a research group running simulations for an upcoming publication.

In this assignment you will not need to virtualize your own hosts, since we make the convention that each individual physical host supports a single VM or virtual host; Mininet takes care of emulating these virtual hosts. It is assumed that each application is pre-installed in each host. The goal is to become familiar with how the virtualized data center network supports and orchestrates the applications running on top of its infrastructure.

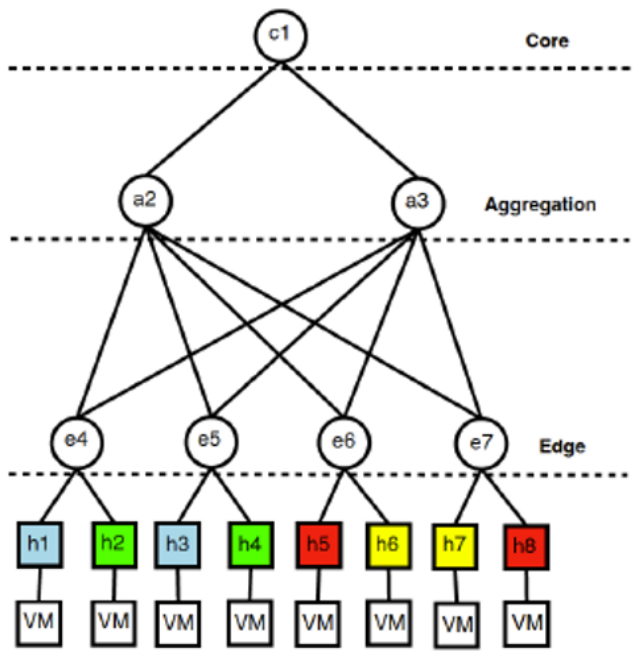

Figure 1: Sample layered data center topology

A properly designed data center network should efficiently route traffic across its rich interconnection fabric, satisfy isolation constraints, and provide VM migration capabilities for its tenants, among other requirements.

- For example, routing strategies could exploit the rich topology of the data center, which contains multiple redundant paths between any pair of hosts, and choose the

shortest path (in terms of number of hops) thus minimizing the allocated bandwidth on the network links.

- According to the **isolation requirement**, VMs belonging to the same tenant offering the same service should be able to communicate only with each other. **Inter-VM communication between different tenants should be blocked** for security reasons, as tenants generally require to operate within their private broadcast domains.
- Regarding the **migration part**, if a tenant's host is planned to shut down for maintenance, the hosted VM has to move into a different host, i.e., migrate, so as to continue offering the service. Such VM transfers have to be done with **minimal reconfiguration and service disruption**.
- For the exercise, you only need to take into account how the network treats the migration of traffic, since the migration should be **transparent on the IP layer**.

In the following sections we describe the steps towards satisfying the aforementioned requirements in your network topology, using the network controller. **Part A** will take you through the processes of building the network topology, performing shortest path routing and implementing and configuring a firewall application. **Part B** will guide you during the implementation of a VM host migration mechanism.

Part A

2-1- Prerequisites

In this exercise, we use the **POX OpenFlow controller** to implement our SDN network applications. The **POX carp branch** is used. We use the **Discovery component of POX** for dynamic topology discovery, and the **Python package NetworkX** for graph-related operations, such as finding suitable paths.

Required Tools:

- Mininet
- POX (carp branch)
- NetworkX
- Python 3.x (ensure POX compatibility—some versions require Python 2.7)

Installation (example on Ubuntu/Kali):

```
sudo apt update
sudo apt install mininet python3-pip
git clone https://github.com/noxrepo/pox
pip3 install network
```

Run POX controller:

```
cd pox
./pox.py log.level --DEBUG openflow.discovery
```

2-2- Understanding the code (Step 0)

Go through the scripts **"clos_topo.py"** and **"CloudNetController.py"**. The first script is for building the emulated network topology on Mininet. The second script contains the code for the SDN applications. The parts you need to fill in are marked with comments such as: #WRITE YOUR CODE HERE.

2-3- Network Topology (Step 1)

Data centers are usually structured in a **tree-like fashion** consisting of three layers of switches: **core, aggregation, and edge/access** layers. The first step is to create a **Clos topology**.

- The number of core switches (tree roots) and the fanout (number of child switches per parent switch) should be configurable parameters.
- In this Clos-like topology, each switch of both the aggregation and edge layers is connected to **all the switches of the previous upper layer**.
- Figure 2 illustrates an example with **2 core switches and fanout equal to 2**.

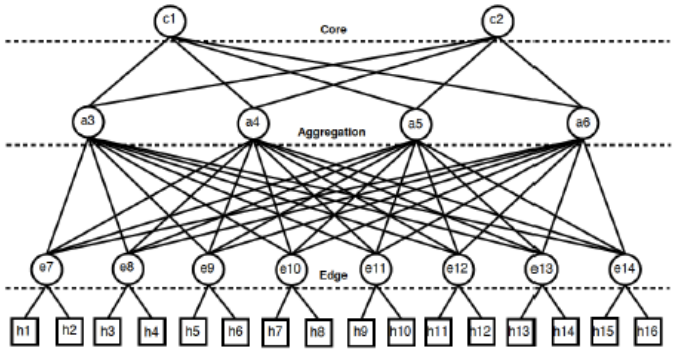

Figure 2: Clos topology with 2 core switches and fanout = 2

2-4- Shortest Path Routing (Step 2)

Your second task is to build a network controller application that forwards traffic along **Shortest Paths**.

- Your application should **calculate and store all the shortest paths of equal length** between any two end-points.
- Afterwards, it should **randomly choose one of these paths** to forward the traffic.
- You will use the **NetworkX Python package** for graph operations. Specifically, the all_shortest_paths(G, source, target) function is a hint.
- **Topology Discovery:** The node adjacency information is gathered dynamically using **POX's Discovery component** via LLDP messages. The shortest path calculation is performed after each link discovery event. Your code must handle the **NetworkXNoPath exception** when the network is disconnected.
- **Flow Rule Installation:** When a new IP flow is triggered, the controller chooses a shortest path and then installs the corresponding flow rules to all switches along the selected path.
    - Installation is performed in an **inverse fashion** (from destination switch up to the source switch) to limit PacketIn occurrences.
    - The function to complete is **install_end_to_end_IP_path**.
    - Flow rules should match the **IP protocol** before matching source/destination IP addresses.
    - Set a dynamic flow expiration with a **10 sec idle timeout**.
- **Packet Handling:** The PacketIn events that trigger the flow installation are handled separately with **PacketOuts** on the source switch using the send_packet function.
- **Initial Flooding/ARP Proxy:** You must fill in the missing code in the SwitchWithPaths class, specifically **flood_on_switch_edge** and **send_arp_reply** functions. To avoid loops, flood only on ports connected to hosts. The controller acts as an **ARP proxy**, replying directly to ARP requests instead of flooding the entire network.

2-4-1 Sanity checks

Initiate the Mininet topology script and wait a bit for the controller to connect to the switches. The script will trigger an initial pingall as a sanity check.

1. Start POX controller:

./pox.py openflow.discovery CloudNetController

2. Run Mininet topology:

sudo python3 clos_topo.py -c 2 -f 2

3. Test connectivity:

Pingall

4. Run the controller using the following command:

Routing Only: ./pox.py openflow.discovery CloudNet Controller --firewall_capability=False --migration_capability=False.

2-5- Firewall Configuration / Isolation (Step 3)

Your next task is to enhance your controller with **isolation capabilities**.

- The firewall should filter packets based on an **access list**. Only packets between hosts of the **same tenant** are allowed to be exchanged.
- This will operate as a **white-list firewall** in both **Layers 2 and 3** (controlling access for both **ARP and IP** packets).
- **Input File:** You must read a **firewall_policies.csv** file.
    - The file format is: Tenant ID,IP1,IP2,....
    - *Sample format for 2 core switches and fanout=2 (16 hosts):*

1,10.0.0.1,10.0.0.3,10.0.0.5,10.0.0.7,10.0.0.9,10.0.0.11,10.0.0.13,10.0.0.15

2,10.0.0.2,10.0.0.4,10.0.0.6,10.0.0.8,10.0.0.10,10.0.0.12,10.0.0.14,10.0.0.16

Each row starts with a distinct tenant ID, followed by a list of IP addresses of hosts assigned to the same tenant. A sample file is available within the assignment code folder. For simplicity, we assume that all odd-numbered hosts (h1, h3, h5....,h15) belong to tenant 1 and all even-numbered hosts (h2, h4, ..., h16) belong to tenant 2. You should place the firewall_policies.csv file under ~pox/ext/ folder. The provided code reads and properly handles the information in the file.

Enforcement: If the source and destination IP addresses do not belong to the same tenant, the controller should install a flow rule to **drop packets** belonging to that micro-flow using the **drop_packets** function (empty action list).

2-5-1- Sanity checks

Initiate the Mininet topology script and wait a bit for the controller to connect to the switches. The script will trigger an initial pingall as a sanity check.

1. Start POX controller:

./pox.py openflow.discovery CloudNetController

2. Run Mininet topology:

sudo python3 clos_topo.py -c 2 -f 2

3. Test connectivity:

Pingall

4. Once you have finished your firewall code, you should check that traffic belonging to the same tenant is routed and forwarded properly, whereas any other inter-tenant traffic, ARP or IP, is blocked. Run your code using the following command:

Firewall Enabled: ./pox.py openflow.discovery CloudNet Controller --firewall_capability=True --migration_capability=False.

For large-scale testing, perform a pingall command in Mininet. All pings among odd-odd and even-even hosts should be successful.

Part B

2-6- Host Migration (Step 5)

Under certain circumstances, **running application VMs have to move from one host (server) to another** within a data center network; this is called **VM migration** (or host migration in this case). The migration process needs to be **transparent on the IP layer** (TCP sessions should not break; IP addresses should be preserved).

- **Scenario:** Host 1 (e.g., tenant A) migrates to an initially inactive host 5, which belongs to the same tenant. Host 5 adopts the IP of the old host 1, and host 1 becomes inactive.
- **Input File: migration_events.csv**
    - Format: Time_to_trigger (secs), Old_Host_IP, New_Host_IP.

    - *Sample:* 180,10.0.0.1,10.0.0.5.

- **Triggering:** When the migration event is triggered, all switch flow rules that relate to flows towards the **old destination IP of host 1 are deleted** to "force" the migration (HINT: handle_migration function). This generates PacketIn events for new IP flows destined to the old IP.

- **Traffic Redirection and Header Rewriting:** The controller must install rules that redirect IP traffic destined for host 1 towards host 5, while the communicating host still believes it is talking to host 1 (same IP).

    - **Forward (host X to host 1) and Reverse (host 5 to host X) path installation and packet header rewriting are needed** to enable the migration on the IP layer.

    - The controller should act similarly to the shortest path routing (HINT: install_migrated_end_to_end_IP_path function), but must also instruct the switch to **rewrite packet headers** (similar to the "Transparent Load Balancer" assignment).

    - The **headers should only be rewritten at the switch that raised the corresponding PacketIn**.

    - You will need to use install_forward_migration_rule, install_reverse_migration_rule, send_forward_migrated_packet, and send_reverse_migrated_packet.

    - **Result:** IP traffic towards the old host (1) should transparently migrate to the new host (5) (with the new destination IP/MAC after rewrites), and traffic stemming from the new host (5) should be masked as if it originated from the old host (1) (using the old source IP/MAC after rewrites).

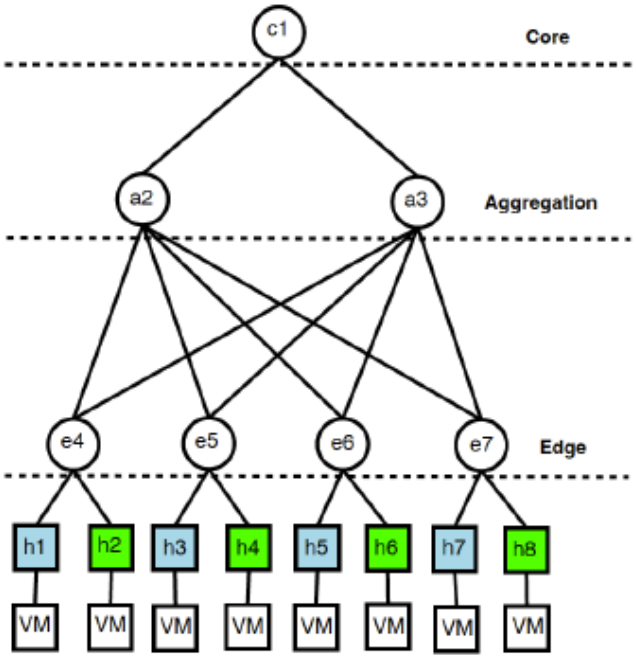

Figure 3: Data center topology with VMs before migration event. Hosts 1, 3, 7 and serve the application of tenant A, while Host 5 from the same tenant is inactive.

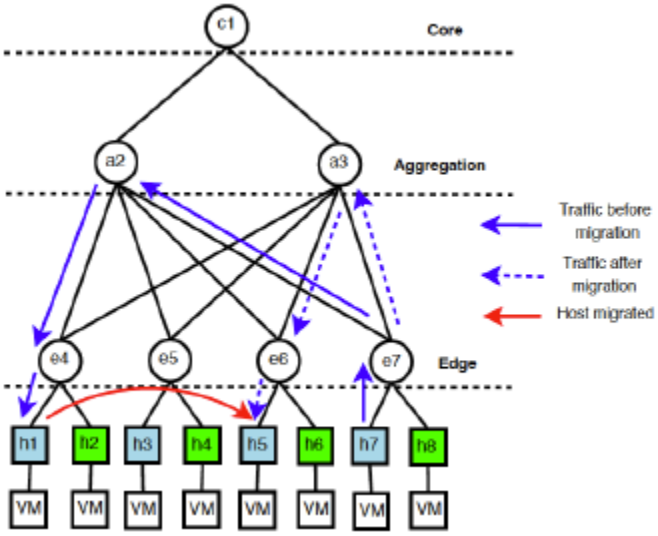

Figure 4: Data center topology with VMs after migration event. (Image showing host h1 has moved to h5. Traffic from host h7 destined to the IP address of h1 is transparently redirected to h5. Arrows illustrate traffic flow before and after migration.)

2-6-1- Sanity checks

Initiate the Mininet topology script and wait a bit for the controller to connect to the switches. The script will trigger an initial pingall as a sanity check.

1. Start POX controller:

./pox.py openflow.discovery CloudNetController

2. Run Mininet topology:

sudo python3 clos_topo.py -c 2 -f 2

3. Test connectivity:

Pingall

4. You should check that the transparent migration properly works. Run your code using the following command:

Migration/Unified: ./pox.py openflow.discovery CloudNet Controller --firewall_capability=False --migration_capability=True.

Deliverables

1. **Controller Implementation Files**
    - Completed CloudNetController.py with all required functions implemented:
    - Updated clos_topo.py
2. **Execution Evidence (Screenshots)**
    - Mininet topology running (pingall output)
    - Demonstration of shortest path routing (flow table or logs)
    - Firewall enforcement:
        - Allowed (same-tenant communication)
        - Blocked (cross-tenant communication)
    - VM migration:
        - Before migration (normal communication)

        - After migration (transparent redirection)

3. **Flow Table Verification**
    - Screenshots or outputs of *dpctl dump-flows* showing:
        - Installed forwarding rules
        - Drop rules for firewall
        - Header rewriting rules for migration
4. **Submit your report in the following format:**
    - **Filename:** Lab7-FirstName-Lastname-StudentNo.PDF

References

[1] Clos Topology. http://en.wikipedia.org/wiki/Clos_network.

[2] Mininet Introduction: Creating Topologies. https://github.com/mininet/mininet/wiki/Introduction-to-Mininet#creating.

[3] Mininet Python API Reference Manual. http://mininet.org/api/.

[4] Mininet Walkthrough. http://mininet.org/walkthrough/.

[5] NetworkX. https://networkx.github.io/.

[6] POX Discovery Component Overview. https://openflow.stanford.edu/display/ONL/POX+Wiki#POXWiki-openflow.discovery.

[7] POX Discovery Component Source Code. https://github.com/noxrepo/pox/blob/carp/pox/openflow/discovery.py.

[8] POX Repository: Carp Branch. https://github.com/noxrepo/pox/tree/carp/.

[9] POX Wiki. https://openflow.stanford.edu/display/ONL/POX+Wiki.

[10] Turnin User Guide. http://www.csd.uoc.gr/services/useful-info/use-the-turnin.html.

Grading Rubric: Lab 7 – SDN Application Development

Category	Criteria	Max Points
1. Network Topology (Step 1)	Correct implementation of a **multi-rooted Clos topology**. Scripts must support configurable core switches and fanout. Switches must be interconnected across layers (Core, Aggregation, Edge) as per requirements.	**15**
2. Shortest Path Routing (Step 2)	Successful integration of **NetworkX** to calculate all equal-length shortest paths. Implementation of **randomized path selection** for load balancing.	**15**
3. Flow & Packet Handling	Flow rules are installed in **inverse order** (Destination $\to$ Source). Correct implementation of **ARP Proxy** and flood_on_switch_edge to prevent broadcast loops. Use of 10s idle timeouts.	**15**
4. Firewall & Isolation (Step 3)	Correct parsing of firewall_policies.csv. Implementation of a **white-list** that permits only intra-tenant traffic. Inter-tenant traffic (both ARP and IP) must be explicitly dropped via flow rules.	**15**
5. VM Migration (Step 5)	Successful trigger of migration from migration_events.csv. Existing flows for the old IP are correctly flushed. Traffic is successfully redirected to the new host.	**10**
6. Header Rewriting (Part B)	**Transparent migration:** Controller correctly rewrites MAC and IP headers at the edge switch. Traffic to the old IP reaches the new host, and responses are masked to appear from the old IP.	**10**
7. Validation & Sanity Checks	Successful execution of pingall under different modes (Routing only, Firewall enabled, Migration enabled). Demonstration that TCP/ICMP sessions do not break during migration.	**10**
8. Documentation & Code Quality	Code is well-commented in the #WRITE YOUR CODE HERE sections. A brief report or README is provided justifying the design choices (e.g., why reverse flow installation is used).	**10**

Best of Luck!